

Database Management Systems

Today Topics

- View in SQL
- User defined Functions
- Stored Procedures

MySQL Views

- In SQL, a view is a virtual table based on the result-set of an SQL statement.
- A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.
- You can add SQL statements and functions to a view and present the data as if the data were coming from one single table.
- A view is a database object that has no values. Its contents are based on the base table.

MySQL Views

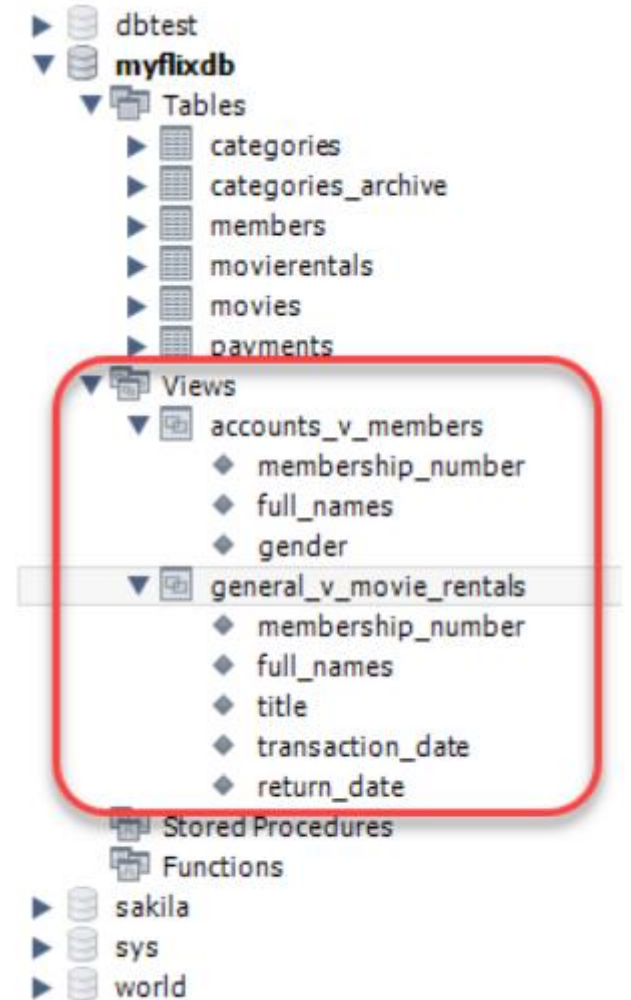
- It contains rows and columns similar to the real table.
- In MySQL, the View is a **virtual table** created by a query by joining one or more tables.
- It is operated similarly to the base table but does not contain any data of its own.
- The View and table have one main difference that the views are definitions built on top of other tables (or views). If any changes occur in the underlying table, the same changes reflected in the View also.

View Syntax

- `CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;`
- OR
- `CREATE [OR REPLACE] VIEW view_name AS
SELECT columns
FROM tables
[WHERE conditions];`

Example

- **CREATE VIEW** accounts_v_members
AS
SELECT
membership_number,
full_names,
gender
FROM members;



Alter | DROP View

ALTER VIEW view_name **AS**

SELECT columns

FROM table

WHERE conditions;

DROP VIEW [IF EXISTS] view_name;

Delimiter

- A MySQL client program such as MySQL Workbench or mysql program uses the delimiter (;) to separate statements and executes each statement separately.
- However, a stored procedure consists of multiple statements separated by a semicolon (;).
- If you use a MySQL client program to define a stored procedure that contains semicolon characters, the MySQL client program will not treat the whole stored procedure as a single statement, but many statements.

Delimiter

- Therefore, you must redefine the delimiter temporarily so that you can pass the whole stored procedure to the server as a single statement.
- To redefine the default delimiter, you use the DELIMITER command:

- Syntax

DELIMITER delimiter_character

Delimiter

- The delimiter_character may consist of a single character or multiple characters e.g., // or \$\$.
- However, you should avoid using the backslash (\) because it's the escape character in MySQL.
- For example, the following statement changes the current delimiter to //:
- DELIMITER //
- Once changing the delimiter, you can use the new delimiter to end a statement as follows:
DELIMITER //
SELECT * FROM customers //
SELECT * FROM products //
- To change the delimiter to the default one, which is a semicolon (;), you use the following statement: DELIMITER ;

User Defined Function

- In MySQL, a function is a stored program that you can pass parameters into and then return a value.
- Just as you can create functions in other languages, you can create your own functions in MySQL.
- The **CREATE FUNCTION** statement is used for creating a stored function and user-defined functions.
- A stored function is a set of SQL statements that perform some operation and return a single value.

User Defined Function

- Just like Mysql in-built function, it can be called from within a Mysql statement.
- By default, the stored function is associated with the default database.
- The **CREATE FUNCTION** statement require **CREATE ROUTINE** database privilege.

User Defined Function

- **CREATE FUNCTION**

function_name [(parameter datatype [, parameter datatype])]

RETURNS return_datatype

BEGIN

declaration_section

executable_section

END;

User Defined Function

function_name

The name to assign to this function in MySQL.

parameter

One or more parameters passed into the function. When creating a function, all parameters are considered to be **IN parameters** (not OUT or INOUT parameters) where the parameters can be referenced by the function but can not be overwritten by the function.

return_datatype

The data type of the function's return value.

declaration_section

The place in the function where you [declare local variables](#).

executable_section

The place in the function where you enter the code for the function.

User Defined Function

```
DELIMITER //  
CREATE FUNCTION SquareofNum (val1 INT )  
RETURNS INT  
BEGIN  
    DECLARE seq INT;  
    SET seq = val1 * val1 ;  
RETURN seq;  
END;  
DELIMITER // ;
```

User Defined Function

- When you want to use the function use it with the simple select statement.
- `Select SquareofNum(3)`

Example

```
CREATE FUNCTION CalculateAmount(userid INT) RETURNS  
float(10,2)  
BEGIN  
    DECLARE totalAmount FLOAT;  
    SELECT SUM(amount) INTO totalAmount FROM credit_user  
WHERE id =userid;  
    RETURN totalAmount;  
END
```

Example

```
DELIMITER $$  
CREATE FUNCTION Func_Calculate_Age ( Age date )  
RETURNS INT DETERMINISTIC  
BEGIN  
DECLARE TodayDate DATE;  
SELECT CURRENT_DATE() INTO TodayDate;  
RETURN YEAR(TodayDate) - YEAR(Age);  
END$$  
DELIMITER ;
```

Example

```
SELECT EmployeeId, Name, Salary, DOB,  
Func_Calculate_Age(DOB) AS Age  
FROM Employee  
WHERE Func_Calculate_Age(DOB) > 30;
```

MySQL Stored Procedure

- Often, stored procedures have parameters.
- The parameters make the stored procedure more useful and reusable.
- A parameter in a stored procedure has one of three modes: IN,OUT, or INOUT.
- MySQL stored procedures are pre-compiled SQL statements stored in a database. They are subroutines containing a name, a parameter list, and SQL statements.
- All relational database systems support stored procedures and do not require any additional runtime-environment packages.

MySQL Stored Procedure

- To invoke stored procedures, you can use the CALL statement or other stored procedures. The first time a stored procedure is invoked, MySQL looks it up in the database catalog, compiles the code, places it in the cache memory, and executes it.

MySQL Stored Procedure

- Today we will cover the basics of the stored procedure that includes the following
 1. Summary of MySQL Stored Procedure
 2. Create a stored procedure using Query and MySQL workbench
 3. Create a Parameterized stored procedure
 4. Drop the Stored Procedure using query and MySQL workbench
- The stored procedure is SQL statements wrapped within the ***CREATE PROCEDURE*** statement. The stored procedure may contain a conditional statement like IF or CASE or the Loops. The stored procedure can also execute another stored procedure or a function that modularizes the code.

MySQL Stored Procedure

- Following are the benefits of a stored procedure:
 - 1. Reduce the Network Traffic:** Multiple SQL Statements are encapsulated in a stored procedure. When you execute it, instead of sending multiple queries, we are sending only the name and the parameters of the stored procedure
 - 2. Easy to maintain:** The stored procedure are reusable. We can implement the business logic within an SP, and it can be used by applications multiple times, or different modules of an application can use the same procedure. This way, a stored procedure makes the database more consistent. If any change is required, you need to make a change in the stored procedure only
 - 3. Secure:** The stored procedures are more secure than the AdHoc queries. The permission can be granted to the user to execute the stored procedure without giving permission to the tables used in the stored procedure. The stored procedure helps to prevent the database from SQL Injection

Procedure Syntax

- `Create Procedure [Procedure Name] ([Parameter 1], [Parameter 2], [Parameter 3])
Begin
SQL Queries..
End`

Procedure Syntax

- In the syntax:
 - 1.The name of the procedure must be specified after the **Create Procedure** keyword
 - 2.After the name of the procedure, the list of parameters must be specified in the parenthesis. The parameter list must be comma-separated
 - 3.The SQL Queries and code must be written between **BEGIN** and **END** keywords

Example

```
DELIMITER //
```

```
CREATE PROCEDURE sp_GetMovies()
```

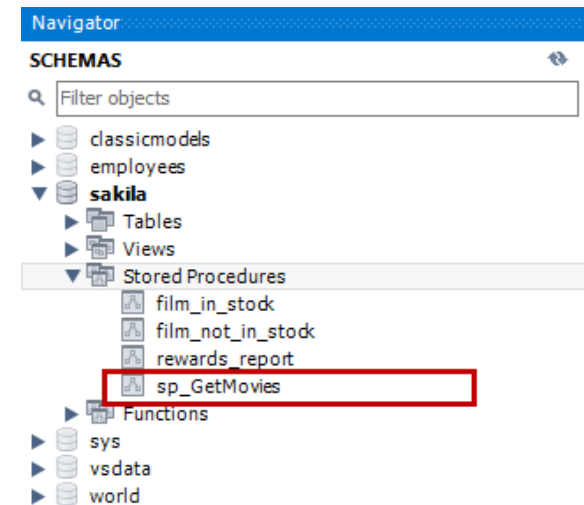
```
BEGIN
```

```
    select title,description,release_year,rating from film;
```

```
END //
```

```
DELIMITER ;
```

- You can view the procedure under stored procedures. See the below screenshot.
- To execute we will call it as
- `CALL sp_GetMovies`



Stored Procedure with Parameter

```
DELIMITER // CREATE PROCEDURE  
procedure_name ( IN | OUT | INOUT parameter_name  
parameter_datatype (length), ... )  
BEGIN  
SQL statements  
END  
// DELIMITER ;
```

Stored Procedure with Parameter

- The parameter modes are:
 - IN – Use to pass a parameter as input. When it is defined, the query passes an argument to the stored procedure. The value of the parameter is always protected.
 - OUT – Use to pass a parameter as output. You can change the value within the stored procedure, and the new value is passed back to the calling program.
 - INOUT – A combination of IN and OUT parameters. The calling program passes the argument, and the procedure can modify the INOUT parameter, passing the new value back to the program.
 -

Stored Procedure with Parameter

```
DELIMITER //  
CREATE PROCEDURE GetCustomerByCountry(IN CountryName  
Varchar(50))  
BEGIN  
SELECT * FROM Customer WHERE Country = CountryName;  
END  
//DELIMITER
```

Stored Procedure with Parameter

- Call `GetCustomerByCountry( 'Lahore' );`

Stored Procedure with Parameter

```
DELIMITER //
```

```
CREATE PROCEDURE sp_CountMoviesByRating(OUT Total_M  
ovies int)
```

```
BEGIN
```

```
    select count(title) INTO Total_Movies from film where rating='P  
G-13';
```

```
END //
```

```
DELIMITER ;
```


Stored Procedure with Parameter

- `CALL sp_CountMoviesByRating(@PGRatingMovies)`
- `Select @PGRatingMovies as Movies`

Example of an INOUT parameter

```
DELIMITER //
```

```
CREATE PROCEDURE sp_CountMoviesByRating_Inout(inout Movies_count int, In param_rating varchar(10))
```

```
BEGIN
```

```
    select count(title) INTO Movies_count from film where rating=param_rating;
```

```
END //
```

```
DELIMITER ;
```

Example of an INOUT parameter

- CALL sp_CountMoviesByRating_Inout(@T,'PG-13');
- Select @T as Movies

Quote of the day



If you focus on **results,**
you will never **change.**

If you focus on **change,**
you will get **results.**

~**Jack Dixon**

